The Theseus Codex

A User Guide

April 2026

The Theseus Codex is the web-facing control plane for the firm's epistemic engine. It is where founders upload the raw materials of thinking — memos, transcripts, essays, live conversations — and where the engine's output comes back to them as a small number of dashboards they can actually act on.

Live at: `https://mqtheseuswork-qiw6.vercel.app`

Contents

1 What the Codex is (and is not) 2

2 The lifecycle of an idea 2

3 The nav: seven destinations 3

4 Every page, explained 3

4.1 Workspace . . . 3
4.2 Review: where the firm argues with itself . . . 4
4.3 Library: what the rest of the world has said . . . 5
4.4 Publication: the firm's public voice . . . 6
4.5 Ops: operator and audit surfaces . . . 6

5 Four workflows, start to finish 7

5.1 "I want to capture and digest a 90-minute conversation." . . . 7
5.2 "I want to publish the firm's position on X." . . . 8
5.3 "I suspect a conclusion is outdated." . . . 8
5.4 "I want to know where the firm stands relative to someone else." . . . 8

6 Glossary 8

7 Appendix: data flow 10

1 What the Codex is (and is not)

The Codex is the screen that founders look at. Everything it shows has been produced by two processes that run **outside** the browser tab: Noosphere (a Python engine that extracts claims and detects contradictions) and Dialectic (a desktop recorder that transcribes live conversations and auto-uploads the transcripts). The Codex itself does not reason; it renders. Thinking of it this way dissolves most of its apparent complexity — almost every page is a view over a specific table in the Postgres database.

It is *not*:

- A content management system. You cannot write posts directly in the Codex. Uploads flow in from the outside.
- A chat interface. The Codex does not run an LLM conversation.
- A publication platform. The **Publication** page prepares material for a separate public site; it does not publish anything itself.
- A social network. There are no comments, threads, likes.

What it *is*: a seven-tab dashboard for the firm's communal brain, with consistent navigation and a one-line explanation at the top of every page.

2 The lifecycle of an idea

Understanding the Codex is easiest as a timeline. A founder has a thought — in a recorded conversation, in a written memo, in a PDF they read. Here is what happens to it, step by step.

1. **Capture.** The idea enters the system either through Dialectic (live audio, auto-transcribed to a JSONL file and auto-uploaded) or through the **Upload** page (drag-and-drop any text, markdown, or transcript file).
2. **Ingest.** Noosphere reads the file and extracts *claims* — short, declarative, falsifiable statements. Each claim carries an author, a topic, an embedding vector, and a provenance chain linking it back to the source text.
3. **Coherence evaluation.** Every new claim is scored against the existing pool on six orthogonal *layers*: natural-language inference, formal argumentation, probabilistic consistency, embedding geometry, information-theoretic compressibility, and an LLM judge. Pairs with a strongly negative aggregated score become **Contradictions**; pairs that the layers disagree about become **Layer review** items.
4. **Synthesis.** Periodically, Noosphere synthesizes firm-level *Conclusions* from groups of mutually supporting claims. Each Conclusion gets one of three tiers: `open` (tentative, dissent remains), `founder` (one founder stands behind it), `firm` (institutional position).
5. **Adversarial stress.** For firm-tier Conclusions, the system generates structured objections (**Adversarial challenges**) and runs the same six-layer stack against the objection. Challenges that score high and go unaddressed can force a demotion back to `open`.

6. **Calibration.** Any Conclusion that makes a falsifiable prediction enters the **Calibration** scoreboard. Resolution dates tick past; outcomes are scored against the predicted probability; Brier / log-loss accumulate by author and domain.

7. **Publication (optional).** A firm-tier Conclusion can be pushed into the **Publication** queue for internal review. Once approved, it is exported into the firm's public-facing static site with a stable slug, versioned URL, and citation metadata.

Everything in the Codex is a window onto one or more stages of this pipeline.

3 The nav: seven destinations

After consolidation, the top navigation has seven items. Four of them are *group labels* — clicking one takes you to a default page, and a sub-navigation row appears below the top nav showing the other pages in that group. You never have to hunt for the right URL.

Top nav	Default page	What lives inside
Dashboard	`/dashboard`	Single page. Daily summary of recent uploads, new conclusions, drift events.
Upload	`/upload`	Single page. Drag-and-drop ingestion form.
Conclusions	`/conclusions`	List of firm claims; click any for a four-tab detail view.
Review	`/contradictions`	Five sub-tabs: *Contradictions*, *Open questions*, *Adversarial*, *Layer review*, *Calibration*.
Library	`/voices`	Four sub-tabs: *Voices*, *Literature*, *Reading queue*, *Research*.
Publication	`/publication`	Single page. Internal review queue + static-site export.
Ops	`/provenance`	Seven sub-tabs: *Provenance*, *Eval runs*, *Post-mortem*, *Decay*, *Rigor gate*, *Methods*, *Founders*.

Every page, regardless of group, has a consistent banner at the top: page name, one sentence on what it shows, one sentence on when you would use it. That banner is the orientation layer — nothing below it will make sense if you skipped it.

4 Every page, explained

The per-page guide below walks every destination in the order a new founder would meet them: workspace first, then review, then library, then the operational tabs at the bottom.

4.1 Workspace

Dashboard `/dashboard`

Shows. Three blocks side by side: the twelve most recent uploads (with status badges for pending / processing / ingested / failed), the eight newest conclusions (with confidence tier + topic hint), and the six most recent drift events.
Use it when. You open the Codex for the day. Nothing here is actionable on its own — it is a starting point that links deeper.

Upload `/upload`

Shows. A single drag-and-drop form.
Use it when. You want to feed a file into the engine manually. Dialectic uploads transcripts automatically; you only use this page for written material or PDFs.
Supported formats. `.md`, `.markdown`, `.txt`, `.vtt`, `.jsonl` are ingested inline (stored as text in Postgres, processed immediately). PDFs, DOCX, and audio files require a binary-storage backend that is not enabled by default on Vercel; they return a clear 413 error with a message pointing at the configuration work still to do.

Conclusions `/conclusions`

Shows. A reverse-chronological list of firm Conclusions, filterable by tier (`firm` / `founder` / `open`) and searchable by topic hint.
Use it when. You want to review what the firm believes right now. Each row links to the Conclusion's detail page.
Temporal replay. The bar at the top of the list lets you set an `asOf` date and see what conclusions would have been consistent with evidence available on or before that date. Replay is a *read projection*, not a literal time machine — it does not re-run the encoder at historical model versions.

Conclusion detail `/conclusions/[id]`

Shows. Everything about a single Conclusion, across four tabs:

- *Overview*: the Conclusion text, rationale, confidence, supporting principles / evidence chain / dissent counts.
- *Provenance*: the extraction chain — how raw text became this claim, with method + confidence per hop.
- *Cascade*: the downstream tree — what other claims depend on this one, recursively. Useful before retracting.
- *Peer review*: founder-cast reviews (endorse / challenge / abstain). You can run a new review here.

Use it when. You need to decide whether to trust, cite, or retract a Conclusion.

4.2 Review: where the firm argues with itself

Contradictions `/contradictions`

Shows. Claim pairs the coherence engine flagged as incompatible, sorted by severity (0–1). Each row shows the two statements, the severity score, and a JSON breakdown of the six layer verdicts.
Use it when. You want the strongest cross-cutting tensions surfaced. Severity above 0.7 is worth a founder look; below 0.5 is usually noise.

Open questions `/open-questions`

Shows. Tensions the system identified but could not resolve automatically. Each item links the two competing claims and summarizes which layers disagreed and why.

Use it when. Planning a deliberate-practice session. Open questions are candidates for explicit discussion rather than automated resolution.

Adversarial `/adversarial`

Shows. Structured objections the system generated against firm-tier Conclusions, with coherence scores against the original claim. Challenges are in one of four states: `pending`, `addressed` (overridden as answered), `fallen` (unaddressed, forces conclusion demotion), `fatal` (human-flagged terminal).
Use it when. You want to stress-test institutional positions. The override action on each row is the primary interaction: `addressed` if the firm has already answered this line of attack in a memo, `fatal` if it should force the Conclusion to the `open` tier.

Layer review `/q/review`

Shows. Coherence disputes where the six layers disagreed with each other. The aggregator had to break a tie; your human verdict is solicited to catch cases where it broke wrongly.
Use it when. You see a Conclusion whose rationale does not match your intuition, and you want to check whether the layers are unanimous or split.

Calibration `/scoreboard`

Shows. Your firm's prediction track record, broken down by author and domain: Brier score, log-loss, decile-bin calibration. A second section shows firm Conclusions that have calibration-discounted confidence applied.
Use it when. You want to know whether you should trust your own firm's probability estimates. Predictions in $[0.45, 0.55]$ are excluded as honest uncertainty — they are not scored.

4.3 Library: what the rest of the world has said

Voices `/voices`

Shows. External thinkers (non-founders) whose claims the system tracks alongside your own: academics, authors, public intellectuals, anyone whose work you have ingested as a source of claims.
Use it when. You want to locate the firm's position relative to established thought. Click a voice for their full claim list.

Voice detail `/voices/[id]`

Shows. One voice's claims, grouped by topic, with relative-to-firm stance indicators (agree / disagree / silent).
Use it when. Writing about a thinker, or preparing an adversarial session that steelmans a specific perspective.

Literature `/literature`

Shows. External reading material ingested into the evidence pool — local PDFs, arXiv abstracts, etc. Each item shows license status and whether full text is stored.
Use it when. Verifying whether the firm has already ingested a particular paper before recommending it.

Reading queue `/reading-queue`

Shows. Readings the research advisor has suggested, ordered by urgency. Each item is grounded in a specific claim match (you see *why* the advisor thinks this reading is relevant).
Use it when. Allocating reading time. Mark items *done* or *skipped* as you work through them; the advisor uses your progress as signal.

Research `/research`

Shows. Topic- and session-level suggestions for the next founder discussion: what to read, what to discuss, which open question to prioritize.
Use it when. Planning the agenda for a founder meeting.

4.4 Publication: the firm's public voice

Publication `/publication`

Shows. Three panels: the *Queue* (Conclusions enqueued for internal review), the *Reviews* (in-progress and completed), and a *Export* button that produces the JSON bundle consumed by the public static site.
Use it when. Moving a firm-tier Conclusion from internal-only to public-facing. Every publish requires an explicit checklist pass (meta-analysis, adversarial engagement, clarity, no leakage, exit conditions declared). The review is the last line of defense against accidental disclosure.

4.5 Ops: operator and audit surfaces

These pages support engineers and operators maintaining the firm's epistemic pipeline. Founders touch them less often than the pages above.

Provenance `/provenance`

Shows. Extraction chain for every claim, with extraction method, confidence, and source pointer. Exportable as CSV / JSON.
Use it when. Auditing how a specific claim entered the store, or debugging an extractor regression.

Eval runs `/eval` and `/eval/runs/[runId]`

Shows. Benchmark runs over the coherence + synthesis pipeline against gold-standard fixtures. The runs list shows macro-F1 and per-domain scores; drilling into a run shows per-case diffs between expected and observed verdicts.
Use it when. After a model upgrade, to confirm nothing regressed before rolling out.

Post-mortem `/post-mortem`

Shows. Conclusions that were later retracted, annotated with what the engine originally believed and the evidence that forced the retraction.
Use it when. You want a sober read of the firm's failure modes before betting on a tier-2 Conclusion in a high-stakes context.

Decay `/decay`

Shows. Firm Conclusions whose supporting evidence has gone stale: embeddings drifted after an encoder upgrade, claims were superseded, model version no longer matches the fixture.
Use it when. You want to prevent bit-rot in the firm's stated positions. The *Revalidate* button re-runs the six-layer stack under the current encoder and updates the Conclusion's decay state.

Rigor gate `/rigor-gate` and `/rigor-gate/[submissionId]`

Shows. Publication submissions requiring a rigor-gate check (decorator verification, meta-analysis, adversarial engagement, clarity checklist). Submit new items via the form; click any pending item to see the reviewer checklist.
Use it when. Gating anything about to leave the firm (publication, external response, public writing).

Methods `/methods`, `/methods/candidates`, `/methods/[name]/[version]`

Shows. Registered coherence / synthesis methods the pipeline uses, with their signatures and provenance. Candidates are proposed-but-unreleased variants. Clicking a method version opens its detail page with *Package* (MIP bundle) and *Document* (method card) generation buttons.
Use it when. Auditing the firm's methodology for external review, or promoting a candidate method into production.

Founders `/founders`

Shows. Co-founders of the firm with their upload and claim contributions, including topic breakdowns.
Use it when. Reviewing who has said what, where the firm's intellectual weight is concentrated.

5 Four workflows, start to finish

Abstract descriptions of pages only go so far. Here are four concrete workflows that exercise the Codex end-to-end.

5.1 "I want to capture and digest a 90-minute conversation."

1. Open Dialectic on your laptop. Start a session. Have the conversation.
2. Press Stop. Dialectic auto-uploads the transcript to the Codex (assuming `DIALECTIC_CLOUD_URL` and `DIALECTIC_CLOUD_API_KEY` are set).
3. In the Codex, go to **Dashboard**. The new upload appears in the Recent uploads panel. Wait for its status to become `ingested`.
4. Go to **Conclusions**. Filter by `topic: <your conversation's topic>` to see newly synthesized firm claims.
5. Click into any surprising conclusion. Use the *Cascade* tab to see what else depends on it.
6. Return to **Review / Contradictions** to check if anything from the new conversation conflicts with prior positions.

5.2 "I want to publish the firm's position on X."

1. Find the firm-tier Conclusion in **Conclusions**.
2. On its detail page, skim the *Adversarial* tab. Resolve any unaddressed challenges (override as `addressed` with pointer to memo, or retract the Conclusion if any challenge is `fatal`).
3. In **Review / Calibration**, check the author's calibration for this domain. If it is weak, consider demoting the Conclusion's public confidence.
4. Enqueue the Conclusion via **Publication / Queue**.
5. Complete the rigor-gate checklist (meta-analysis, adversarial engagement, clarity, no leakage, exit conditions). Click approve.
6. Click *Export* to download the public JSON bundle; drop it into `theseus-public/content/published.json` and rebuild the static site.

5.3 "I suspect a conclusion is outdated."

1. Go to **Ops / Decay**. Scan for conclusions with high decay scores.
2. Find the conclusion in question; click *Revalidate*. Watch the coherence stack re-run; the decay state updates.
3. If decay is confirmed, open the conclusion detail, go to *Cascade*, and identify everything downstream.
4. If major, discuss in the next founder session and retract via **Publication** (if public) or directly via the adversarial `fatal` override (if internal).

5.4 "I want to know where the firm stands relative to someone else."

1. Go to **Library / Voices**. Click the voice in question.
2. Scan their claims list. Each row shows a stance badge: agree / disagree / silent.
3. Click any `disagree` row to see which firm Conclusion pushes back, and why.
4. If gaps show up in the firm's position (many `silent` on important topics), file a **Library / Research** suggestion for next session.

6 Glossary

Claim A single declarative, falsifiable statement extracted from an upload. The atomic unit of the system. Carries author, topic, embedding, provenance.

Conclusion A firm-level synthesized belief derived from multiple supporting claims. Has one of three confidence tiers.

Confidence tier `open` (tentative, dissent present), `founder` (one founder stands behind it), `firm` (institutional position).

Six-layer coherence Six independent coherence tests applied to every claim pair: NLI, formal argumentation, probabilistic consistency, embedding geometry, information-theoretic compressibility, LLM judge. The aggregator combines them into a single verdict.

Adversarial challenge A structured objection the system itself generates against a firm-tier Conclusion. Survives or falls based on coherence against the Conclusion.

Cascade The transitive dependency tree rooted at one Conclusion: what other claims and conclusions would need to change if this one were retracted.

Provenance The step-by-step extraction chain from raw source text to a structured claim: extractor name, confidence, intermediate transformations.

Drift event An observation that a principle or claim's embedding has shifted significantly after recent uploads. Quantitative signal, not a verdict.

Decay The opposite arrow: a Conclusion whose supporting evidence has stopped matching the current encoder / world / model. Periodic revalidation is the remedy.

Rigor gate A checklist every publication-bound item must pass: methodology declared, adversarial engagement complete, clarity check, no leakage, exit conditions stated.

Method registry Versioned, signed record of the extractors and coherence methods the pipeline uses. Enables reproducibility and external audit.

Calibration-discounted confidence A Conclusion's stated confidence, adjusted by the author's past Brier performance in the same decile bin. Surfaces only when the feature flag is on.

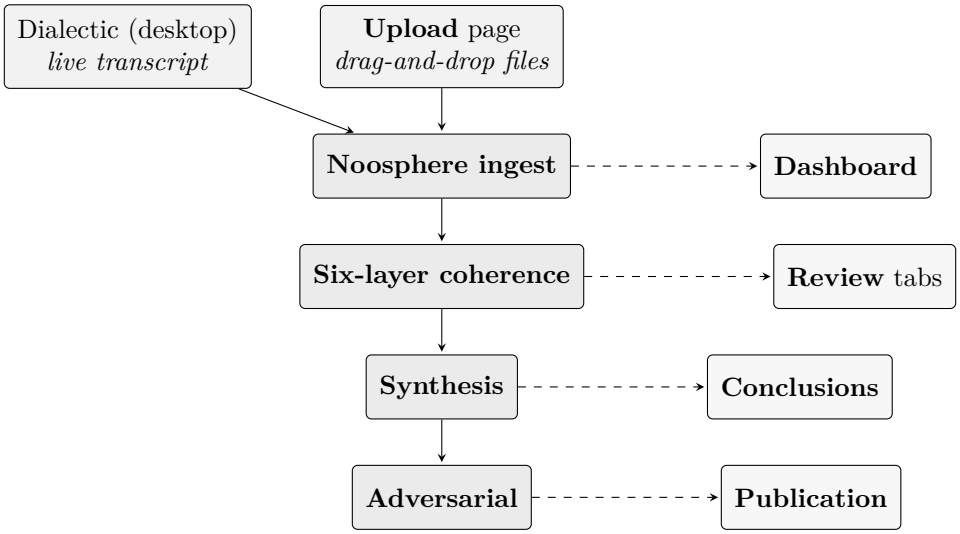
Open question A tension between claims the system raised but could not resolve. Candidate for deliberate-practice discussion.

Voice A non-founder thinker whose claims the system tracks for comparison.

Reading queue Advisor-suggested readings, ordered by urgency, grounded in claim matches.

Temporal replay A read-only projection that shows the Codex as it would have looked given evidence available on a chosen past date. Encoder drift is not undone.

7 Appendix: data flow



Solid arrows are data flowing through the epistemic engine. Dashed arrows are the Codex reading from the database. The Codex never writes into the engine — it only offers the human-in-the-loop buttons (override, approve, retract) that adjust state, while the engine keeps running in the background.